

```

"""
Web Scraper – Quotes Collector
-----

A portfolio project demonstrating real-world web
scraping skills:
- Sending HTTP requests
- Parsing HTML with BeautifulSoup
- Handling pagination (multiple pages)
- Structuring scraped data into a DataFrame
- Exporting to CSV (the deliverable a client pays
for)

Site used: quotes.toscrape.com – a site built
specifically for
practicing scraping, so it's safe and legal to
scrape.

Run this in Google Colab: colab.research.google.com
First run: !pip install requests beautifulsoup4
(Colab usually has these already)
Then paste the rest of this file into a cell and
run.
"""

import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

# -----
# STEP 1: Set up
# -----

BASE_URL = "http://quotes.toscrape.com/page/{}/"
HEADERS = {"User-Agent": "Mozilla/5.0 (portfolio

```

```

project scraper)"}

all_quotes = []

# -----
# STEP 2: Loop through pages and scrape
# -----
page = 1
while True:
    url = BASE_URL.format(page)
    response = requests.get(url, headers=HEADERS)

    if response.status_code != 200:
        print(f"Stopped at page {page} (status {response.status_code})")
        break

    soup = BeautifulSoup(response.text, "html.parser")
    quote_blocks = soup.find_all("div", class_="quote")

    # No more quotes = we've reached the last page
    if not quote_blocks:
        print(f"No more quotes found. Finished at page {page - 1}.")
        break

    for block in quote_blocks:
        text = block.find("span", class_="text").get_text(strip=True)
        author = block.find("small", class_="author").get_text(strip=True)
        tags = [tag.get_text(strip=True) for tag in block.find_all("a", class_="tag")]

        all_quotes.append({
            "Quote": text,

```

```

        "Author": author,
        "Tags": ", ".join(tags)
    })

    print(f"Scraped page {page} –
{len(quote_blocks)} quotes")
    page += 1

    # Be polite: small delay between requests so
    you don't hammer the server
    time.sleep(1)

# -----
# STEP 3: Structure and export the data
# -----
df = pd.DataFrame(all_quotes)

print(f"\n=== TOTAL QUOTES SCRAPED: {len(df)} ===")
print(df.head())

# -----
# STEP 4: Quick analysis (shows you can go beyond
raw scraping)
# -----
print("\n=== TOP 5 AUTHORS BY QUOTE COUNT ===")
print(df["Author"].value_counts().head())

# -----
# STEP 5: Save the deliverable
# -----
df.to_csv("scraped_quotes.csv", index=False)
print("\nData exported to scraped_quotes.csv")

# -----

```

```
-----  
# NOTE FOR REAL CLIENT GIGS:  
# - Always check a site's robots.txt and terms of  
service before scraping  
# - Add try/except around requests to handle  
network errors gracefully  
# - Some sites need Selenium (for JavaScript-  
rendered content) instead  
# of requests + BeautifulSoup – that's a good  
"next level" skill to learn  
# -----  
-----
```